Distinct Routes

A game consists of n rooms and m teleporters. At the beginning of each day, you start in room 1 and you have to reach room n.

You can use each teleporter at most once during the game. How many days can you play if you choose your routes optimally?

Input

The first input line has two integers n and m: the number of rooms and teleporters. The rooms are numbered 1,2,…,n.

After this, there are m lines describing the teleporters.
Each line has two integers a and b: there is a teleporter from room a to room b.

There are no two teleporters whose starting and ending room are the same.

Output

First print an integer k: the maximum number of days you can play the game.
Then, print k route descriptions according to the example. You can print any valid solution.

Constraints

2 <= n <= 500
1 <= m <= 1000
1 <= a,b <= n

Example

Input:
```
6 7
1 2
1 3
2 6
3 4
3 5
4 6
5 6
```
Output:
```
2
3
1 2 6
4
1 3 4 6
```

Problem Breakdown

Lets understand the problem first

We have:
- n rooms (nodes).
- m teleporters (directed edges).
- You start at room 1 and want to reach room n.
- Each teleporter can be used at most once overall (not per day, but total).

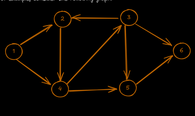
You can think of this as:
"How many edge-disjoint paths exist from 1 to n?"

That means:
Find the maximum number of different routes from 1 → n, such that no teleporter (edge) is reused in two routes.

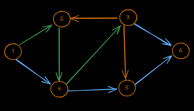
Edge-disjoint paths

We will first focus on the problem of finding the maximum number of edge-disjoint paths from the source to the sink. This means that we should construct a set of paths such that each edge appears in at most one path.

For example, consider the following graph:



In this graph, the maximum number of edge-disjoint paths is 2. We can choose the paths 1 → 2 → 4 → 3 → 6 and 1 → 4 → 5 → 6 as follows:



It turns out that the maximum number of edge-disjoint paths equals the maximum flow of the graph, assuming that the capacity of each edge is one.

After the maximum flow has been constructed, the edge-disjoint paths can be found greedily by following paths from the source to the sink.

Mapping to Flow Network

This maps perfectly to a maximum flow problem:

- Each teleporter is like a directed edge (a → b) with capacity = 1.
- A "unit of flow" from source (1) to sink (n) represents one distinct path.
- Because capacity = 1, each edge can only be used in one path — exactly what we want.

So:
- Source = 1
- Sink = n
- Capacity(a, b) = 1
- Compute the maximum flow → gives the maximum number of edge-disjoint paths.

What do we need after max flow?

Once we know the maximum flow value (say, k),
we must also reconstruct each of the k paths used by the flow.

How?

After computing the max flow, we look at all edges in the residual graph:

- If the reverse edge has capacity = 1 (and forward edge capacity = 0),
  it means that one unit of flow passed through that edge.

So we can repeatedly traverse edges with used flow = 1 from 1 to n, collecting nodes along the way, and mark them as visited (to avoid reusing edges).

That reconstructs each distinct route.

Code

```
#include <bits/stdc++.h>
using namespace std;

struct Edge {
    int to, rev;
    int cap, flow;
};

struct Dinic {
    int n;
    vector<vector<Edge>> adj;
    vector<int> level, it;

    Dinic(int n) : n(n), adj(n), level(n), it(n) {}

    void add_edge(int u, int v, int cap) {
        Edge a = {v, (int)adj[v].size(), cap, 0};
        Edge b = {u, (int)adj[u].size(), 0, 0};
        adj[u].push_back(a);
        adj[v].push_back(b);
    }

    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        q.push(s);
        level[s] = 0;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (auto &e : adj[u]) {
                if (e.cap - e.flow > 0 && level[e.to] == -1) {
                    level[e.to] = level[u] + 1;
                    q.push(e.to);
                }
            }
        }
        return level[t] != -1;
    }

    int dfs(int u, int t, int f) {
        if (u == t) return f;
        for (int &i = it[u]; i < (int)adj[u].size(); i++) {
            Edge &e = adj[u][i];
            if (e.cap - e.flow > 0 && level[e.to] == level[u] + 1) {
                int pushed = dfs(e.to, t, min(f, e.cap - e.flow));
                if (pushed > 0) {
                    e.flow += pushed;
                    adj[e.to][e.rev].flow -= pushed;
                    return pushed;
                }
            }
        }
        return 0;
    }

    int maxflow(int s, int t) {
        int flow = 0;
        while (bfs(s, t)) {
            fill(it.begin(), it.end(), 0);
            while (int pushed = dfs(s, t, 1e9))
                flow += pushed;
        }
        return flow;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    Dinic dinic(n + 1);

    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        dinic.add_edge(a, b, 1);
    }

    int maxPaths = dinic.maxflow(1, n);
    cout << maxPaths << "\n";

    // Extract paths using DFS on residual network
    vector<vector<int>> routes;

    for (int i = 0; i < maxPaths; i++) {
        vector<int> path;
        int cur = 1;
        path.push_back(cur);

        while (cur != n) {
            bool found = false;
            for (auto &e : dinic.adj[cur]) {
                // If there's flow in forward direction, use it
                if (e.flow > 0 && e.to >= 1 && e.to <= n) {
                    e.flow = 0; // Mark this edge as used
                    cur = e.to;
                    path.push_back(cur);
                    found = true;
                    break;
                }
            }
            if (!found) break; // Shouldn't happen if flow is valid
        }
        routes.push_back(path);
    }

    for (auto &path : routes) {
        cout << path.size() << "\n";
        for (int x : path) cout << x << " ";
        cout << "\n";
    }

    return 0;
}
```